

Kernel and Non-Kernel Learning Under Real-World Data Imperfections

A Cross-Domain Study of Data Quality, Geometry, Failure, Bias, and Computation

Graduate Machine Learning — Spring 2026

Abstract

This study asks a deliberately practical question: when does the dataset matter more than the learning algorithm? Four domains are investigated under a common experimental framework but with deliberately different engineering pipelines: multi-city Airbnb listings, New York City 311 service requests, IBM employee attrition records, and online retail transactions. Seven prediction tasks are constructed: Airbnb price and superhost status, 311 complaint category and resolution time, employee attrition, future customer segment, and future customer spending. Before modeling, missingness mechanisms, outliers, redundancy, leakage, label ambiguity, duplicates, censoring, imbalance, temporal ordering, and aggregation are examined. Logistic regression, linear regression, K-nearest neighbors, decision trees, kernel support vector machines, kernel ridge regression, kernel KNN, and kernel PCA followed by logistic regression are implemented from scratch with NumPy and SciPy.

The results reject any universal “kernel methods are better” conclusion. NYC 311 was the only dataset with a positive average kernel benefit: a linear-kernel SVM improved complaint macro-F1 from the best non-kernel value of 0.225 to 0.274, while RBF kernel ridge regression reduced resolution-time MAE from 64.24 to 60.90 hours. Airbnb showed almost no regression gain from a linear kernel and a clear classification advantage for ordinary logistic regression. IBM HR favored logistic regression in macro-F1, although a linear-kernel SVM improved balanced accuracy by accepting more false positives. Online Retail favored logistic regression for segments and a decision tree for spending because zero inflation and extreme tails violated a single smooth kernel surface. The worst errors were dominated by target tails, ambiguous labels, sparse histories, censored operational processes, and underrepresented groups. The study therefore concludes that model geometry matters, but only after the observational unit, target, missingness, leakage boundary, and evaluation design are made valid.

Keywords: kernel methods, data quality, missing data, outliers, SVM, kernel ridge regression, KPCA, KNN, failure analysis, fairness.

1 Introduction

Machine-learning courses often begin with a clean matrix $X \in \mathbb{R}^{n \times d}$, a complete label vector y , and a random train/test split. Under those conditions the central intellectual activity appears to be algorithm selection. Real systems reverse the order. The difficult questions are usually: What is one observation? When is the prediction made? Which fields exist at that moment? Is a missing value an accident, a structural state, or a consequence of the outcome? Does an extreme value represent fraud, a data-entry error, or a valid rare case? Are repeated records duplicates or independent reports of the same event? Does the test set represent the deployment population?

This homework is designed around that reversal. The four datasets are not merely four tables. They are four distinct empirical manifolds:

1. Airbnb is a mixed geographic and economic marketplace. Its feature space combines location, property type, host behavior, availability, reviews, and right-skewed prices.
2. NYC 311 is a high-volume event stream. Complaint labels evolve, operational outcomes are right-censored, reports cluster in time and space, and service channels measure access as well as incidents.
3. IBM HR is a small, unusually complete, imbalanced tabular dataset. Its main risks are variance, weak transportability, and misleading accuracy.
4. Online Retail is not initially a customer table at all. It is a transaction log containing purchases, returns, cancellations, repeated invoices, anonymous buyers, and time.

The central thesis is that an algorithm can only learn the geometry supplied by the data pipeline. If the observational unit is wrong, the target leaks into the predictors, or a future quantity is used to predict itself, a sophisticated kernel only makes the invalid experiment more convincing. Conversely, careful engineering can reveal when nonlinear similarity is useful and when it merely increases variance.

The contributions are:

- a dataset-specific quality investigation of missingness, outliers, redundancy, leakage, and labels;
- seven leakage-controlled prediction tasks derived from

four raw sources;

- NumPy/SciPy implementations of all required linear, distance-based, tree, and kernel algorithms;
- a kernel investigation using predictive performance, centered kernel alignment, effective rank, and similarity concentration;
- instance-level post-mortems of the ten worst predictions for every task;
- group, temporal, geographic, and computational analyses;
- a direct explanation of why textbook benchmarks overestimate production readiness.

This document is written to be read at two levels. At the first level it is a conference-style empirical report. At the second, it is a guided explanation of the mathematical and implementation decisions in the accompanying notebook and Python modules.

2 Dataset Description and Topology

2.1 Airbnb Open Data

The Airbnb source is a merged Inside Airbnb listing export [?]. The local file contains 23,703 listings and 91 raw columns from Nashville (9,443), Seattle (6,996), Dallas (6,211), Bozeman (600), and Albany (453). Thus, the largest city contributes about 39.8% of all observations. A row represents a listing at a scrape time, not a completed reservation.

Two targets are defined:

Price regression: the advertised nightly price. Of 23,703 rows, 19,296 have a positive usable price.

Superhost classification: `host_is_superhost`, mapped to 1/0. There are 22,936 known labels and 767 unknown labels.

The topology is mixed. Latitude and longitude define a continuous spatial manifold, city and neighborhood define nested categorical partitions, room and property types define structural regimes, and reviews and availability represent platform activity. Price is extremely right-skewed: some hotel-room observations are listed near \$50,000 while ordinary private rooms have medians around \$70–\$92. These points cannot be automatically deleted because the data alone do not prove whether they are luxury inventory, calendar artifacts, minimum-stay pricing artifacts, or scrape errors.

A further dependence exists at the host level. One host can own multiple listings, so an ordinary row-level random split can put nearly identical host behavior into training and test sets. The final design therefore carries `host_group_id` only long enough to perform a group-disjoint split. It is removed before modeling.

2.2 NYC 311 Service Requests

The NYC source is the official 311 service-request dataset [?]. The local snapshot contains 3,409,102 records from 2025 and 2026 and occupies approximately 2.8 GB. A row is one report, not necessarily one unique physical incident. Multiple residents may report the same noisy party or blocked driveway; deleting all near-duplicates would erase complaint intensity.

Two targets are defined:

Complaint classification: the 20 most common complaint types plus an `OTHER` category.

Resolution regression: hours from creation to closure for requests with a valid nonnegative closure duration.

The complete complaint distribution is strongly imbalanced. Illegal Parking alone has more than half a million records in the full file, followed by Noise–Residential and HEAT/HOT WATER. Resolution time is a process outcome, not an ordinary continuously observed label. At the snapshot date, open cases have not yet reached their final duration. They are right-censored. Assigning zero to those cases would mean “unresolved” equals “instant resolution,” reversing the semantics.

The raw table is streamed in chunks. During one pass, exact complaint, borough, and month counts are accumulated, while a deterministic hash of `Unique Key` selects a 1% sample. This produces 34,283 classification rows and 32,755 closed-case regression rows. The sample is cached using source file size, modification timestamp, and sample rate. Re-running the notebook therefore does not repeatedly scan 2.8 GB unless the source changes.

Sampling quality is measured by total variation distance

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_a |P(a) - Q(a)|. \quad (1)$$

The sample-to-population TVDs are 0.0174 for complaint, 0.0038 for borough, and 0.0060 for month. These values are small enough that the sample preserves the main marginal distributions while reducing storage and diagnostic time by roughly two thirds relative to the original 3% sample.

2.3 IBM HR Employee Attrition

The IBM HR Analytics dataset [?] contains 1,470 employee records, 35 raw columns, no missing cells, and no exact duplicates. Attrition is imbalanced: 237 employees (16.1%) have `Attrition=Yes`, while 1,233 remain. Three fields are constant: `EmployeeCount`, `Over18`, and `StandardHours`. `EmployeeNumber` is an identifier.

The topology is a low-sample mixed tabular manifold. Many features are ordinal satisfaction scores; others are continuous rates or tenure counts; others are categories such as department, role, travel frequency, overtime, marital status, and education field. Because the sample is small, a flexible boundary can fit idiosyncrasies instead

of stable structure. Because the minority class is small, ordinary accuracy is dominated by the majority.

The dataset is also suspiciously clean for human-resources data. No missing values, perfectly standardized fields, and a commonly cited demonstration origin imply that external validity is a larger risk than imputation. The report therefore treats discovered relationships as predictive associations within this sample, not causal laws of employee behavior.

2.4 Online Retail Transactions

The UCI Online Retail dataset [?] contains 541,909 transaction lines from 1 December 2010 through 9 December 2011. Each row contains invoice, product, quantity, time-stamp, unit price, customer ID, and country. There are 5,268 exact duplicates, 135,037 rows without customer ID, 9,288 cancellation-invoice rows, 10,624 nonpositive quantities, and 2,517 nonpositive unit prices.

The raw observational unit is unsuitable for the assignment targets. A transaction line cannot be labeled with “future customer spending” without repeating the same customer across many rows and leaking customer history between splits. The correct transformation is from event lines to one historical row per customer.

The cutoff is 1 September 2011. Transactions before the cutoff create historical features. Transactions from the cutoff through 9 December 2011 define:

Future spending: positive future net spending, computed as future purchases minus returns and clipped at zero.

Future segment: `Future_Inactive` for zero future net spending and low/mid/high tertiles among active customers.

There are 3,317 customers with historical purchases. The resulting segment counts are 1,374 inactive, 648 high, 648 low, and 647 mid. Future net spending has median \$196.80, 99th percentile \$9,623.74, and maximum \$129,120.07. This is a zero-inflated, heavy-tailed target rather than a Gaussian response.

3 Data Cleaning and Feature Engineering

3.1 General leakage discipline

Leakage occurs when a predictor contains information that would not exist at prediction time or is a direct transformation of the target. The following fields are explicitly excluded:

- Airbnb price quote fields and estimated revenue from price regression;
- the raw superhost flag from superhost predictors;
- 311 closure date, status, and resolution description from duration predictors;

- the raw complaint field from the engineered complaint target;
- employee number from HR;
- future orders, future gross spend, future returns, and future net spend from retail predictors.

Imputation, robust scaling, category selection, and one-hot mappings are fit only on training rows. This matters especially for distance and kernel methods. If the median or category vocabulary is computed on the full dataset, the test distribution influences the feature map even if its labels are hidden.

3.2 Missing-data analysis

3.2.1 What MCAR, MAR, and MNAR mean

Let R_j be the indicator that variable X_j is observed. Missing completely at random (MCAR) means

$$P(R \mid X_{\text{obs}}, X_{\text{mis}}) = P(R). \quad (2)$$

Missing at random (MAR) means missingness may depend on observed variables but, conditional on them, not on the missing value:

$$P(R \mid X_{\text{obs}}, X_{\text{mis}}) = P(R \mid X_{\text{obs}}). \quad (3)$$

Missing not at random (MNAR) permits dependence on the unobserved value itself [?].

Observed data can provide evidence against MCAR by showing that R_j is associated with observed fields; formal omnibus MCAR tests follow the same independence principle [?]. They cannot generally prove MAR versus MNAR, because MNAR is a statement about values that are not observed. Accordingly, the analysis does not claim to “prove MNAR.” It reports evidence, domain mechanism, and the assumption under which the engineering decision is valid.

For a missing column j , numerical observed fields are compared between $R_j = 0$ and $R_j = 1$ with a Mann–Whitney U test and rank-biserial effect. Categorical fields use a chi-square independence test and Cramer’s V :

$$V = \sqrt{\frac{\chi^2}{n \min(r-1, c-1)}}. \quad (4)$$

3.2.2 Airbnb missingness

Price missingness is strongly associated with scrape source ($V = 0.822$) and city ($V = 0.398$). Host response missingness is associated with city ($V = 0.844$) and review activity. Review-score missingness is perfectly separated by review count in the sampled test: listings with no usable review history cannot possess review scores. Bedroom missingness depends on room type ($V = 0.310$) and source. These results reject MCAR.

The mechanism is a mixture of MAR and structural absence. A review score is structurally absent for an unreviewed listing. Host-response fields may be absent because

Table 1: Final task inventory. Exact kernel experiments use controlled subsets so that every required from-scratch method is computationally feasible.

Task	Type	Engineered rows	Train	Test	Encoded dimensions
Airbnb price	Regression	19,296	700	300	97
Airbnb superhost	Classification	22,936	800	300	98
NYC complaint	Classification	34,283	420	260	81
NYC resolution	Regression	32,755	650	300	87
HR attrition	Classification	1,470	1,103	367	63
Retail segment	Classification	3,317	800	300	26
Retail spending	Regression	3,317	700	300	25

of host activity or source extraction. Four compact indicators are retained: missing review information, missing host response, missing capacity information, and missing host profile. The original numerical gaps remain until the train-only preprocessor replaces them by training medians. Categorical gaps become a training-defined **Missing** level. Creating one indicator for every review subscore was avoided because those flags were perfectly correlated and inflated distance without adding information.

Price and superhost labels are not imputed. Imputing a target fabricates ground truth and produces artificially confident evaluation.

3.2.3 NYC missingness and censoring

Closure-date missingness has Cramer’s $V = 0.970$ with status, $V = 0.549$ with complaint, and $V = 0.475$ with agency. This is not MCAR. It is operational censoring: open requests are missing a final duration precisely because the process has not ended. The ordinary regression analysis therefore uses closed cases only and states that it estimates duration conditional on observed closure. A full production treatment would use survival analysis, where open cases contribute right-censored likelihood terms.

Coordinate missingness is associated with complaint ($V = 0.526$), agency ($V = 0.362$), channel, and borough. The likely mechanism is MAR: whether a coordinate is available depends on address collection and agency workflow. Borough, city, ZIP, community board, and channel are retained, while numerical coordinate gaps are handled inside training folds.

3.2.4 HR missingness

There are zero observed missing cells. This removes the need for mathematical imputation but does not prove complete measurement. Important latent variables—manager quality, labor-market alternatives, caregiving duties, team climate, and whether an employee was encouraged to leave—are absent by design. “No NaN” is not equivalent to “no missing information.”

3.2.5 Retail missingness

Customer-ID missingness is associated with quantity (rank-biserial effect -0.505), unit price (0.263), country ($V = 0.171$), and cancellation behavior. The records are not MCAR. They may represent guest purchases, system processes, or unmatched transactions. Since customer histories cannot be reconstructed without identity, those lines are excluded from customer-level supervised learning. Assigning a common “unknown customer” ID would merge unrelated people into a fictional mega-customer.

Description missingness is associated with price and quantity but description is unnecessary after customer behavioral aggregation. Therefore no text imputation is performed.

3.3 Outlier investigation

Three methods are compared because they encode different definitions.

Z-score. For a scalar x_i ,

$$z_i = \frac{x_i - \bar{x}}{s}. \quad (5)$$

A conventional flag is $|z_i| > 3$. This is meaningful when the mean and standard deviation summarize the distribution. It is unstable for heavy-tailed price, duration, quantity, and spending, because the same extreme points inflate both $\bar{x}$ and s .

IQR rule. With $\text{IQR} = Q_3 - Q_1$, a point is flagged outside

$$[Q_1 - 1.5 \text{IQR}, Q_3 + 1.5 \text{IQR}]. \quad (6)$$

This is robust but univariate. It cannot distinguish a valid expensive large house from an implausibly expensive private room.

Isolation Forest. Isolation Forest recursively chooses a random feature and random split. Rare points tend to be isolated with short paths $[?]$. If $h(x)$ is average path length and $c(m)$ is the expected path length for a sample of size m , the anomaly score is

$$s(x, m) = 2^{-E[h(x)]/c(m)}. \quad (7)$$

The scratch implementation builds 100 trees on samples of at most 256 rows and flags the top 1% scores.

The disagreement is substantial. For Airbnb price, Z-score flags 20 of 5,000 cases, IQR flags 349, and Isolation Forest flags 50. Z/IQR Jaccard overlap is only 0.057. For 311 resolution time, counts are 113, 749, and 50, with IQR/isolation overlap 0.067. For retail unit price, counts are 3, 448, and 51. These are not minor numerical differences; they show that “outlier” is a modeling definition.

The adopted rule is conservative:

- impossible target values, such as nonpositive Airbnb prices or nonpositive retail prices for spending, are removed;
- plausible positive tails are retained;
- log targets reduce multiplicative skew;
- winsorized/capped alternatives are retained for sensitivity analysis, not silently substituted for truth;
- the ten worst predictions are examined to decide whether tails are data errors or genuine model limitations.

3.4 Feature-quality analysis

Constants are removed because they have zero variance and cannot alter any prediction. HR loses `EmployeeCount`, `Over18`, and `StandardHours`. Airbnb loses `calendar_updated`. The shared-room host-count feature is 99.55% dominated and excluded from the selected feature set.

High Spearman correlations identify redundant information: Airbnb bathrooms versus parsed bathroom text (0.99998), host listings versus total host listings (0.973), HR job level versus monthly income (0.920), retail return count versus return-value rate (0.958), and retail total units versus gross spend (0.922). Redundancy is not automatically removed if two variables carry different operational meaning, but mechanically duplicated summaries are reduced. For example, the final retail table omits purchase-line count, active-day count, raw return value, and units-per-order because retained features already represent those behaviors.

Feature engineering is deliberately domain-specific:

Airbnb: parsed money and percentages; host tenure; listing age; days since last review; amenity count; parsed bathrooms; compact missingness states.

311: hour, day of week, month, day, weekend, business-hours indicator, elapsed calendar time, and cyclic coordinates

$$\sin(2\pi h/24), \quad \cos(2\pi h/24)$$

so that 23:00 and 00:00 are geometrically adjacent. Near-duplicate report count is created from complaint, detail, rounded coordinates, and ten-minute window.

HR: income per job level; company tenure divided by total career; role and manager tenure ratios; promotion stagnation ratio.

Retail: recency, frequency, gross monetary value, prod-

uct diversity, basket size, order value, return count/rate, country, weekend/evening share, two-half spending, and spending trend.

3.5 Engineering mathematics and reproducibility

3.5.1 Train-only numerical imputation

Suppose numerical feature j has observed training values $\{x_{ij} : R_{ij} = 1, i \in \mathcal{T}\}$. The fitted imputation parameter is

$$m_j = \text{median}\{x_{ij} : R_{ij} = 1, i \in \mathcal{T}\}. \quad (8)$$

Transformation is

$$\tilde{x}_{ij} = \begin{cases} x_{ij}, & R_{ij} = 1, \\ m_j, & R_{ij} = 0. \end{cases} \quad (9)$$

The median minimizes the sum of absolute deviations and is therefore less sensitive than the mean to the heavy tails documented in the outlier analysis. It is not claimed to recover the true missing value. It creates a neutral numerical placeholder, while missingness indicators allow the model to learn a different intercept for structurally incomplete records.

For a domain-level missing indicator M_i , a linear predictor becomes

$$\eta_i = \beta_0 + \beta_j \tilde{x}_{ij} + \beta_M M_i + \dots \quad (10)$$

When $M_i = 1$, β_M can correct the systematic shift associated with absence. This is more defensible than pretending that an imputed median is observed truth. The approach still relies on the predictive stability of the missingness pattern; it does not solve unobserved MNAR bias.

3.5.2 Categorical encoding and rare levels

For categorical feature C_j , the training data define a vocabulary $\mathcal{V}_j$ containing frequent levels. A value is mapped by

$$g_j(c) = \begin{cases} c, & c \in \mathcal{V}_j, \\ \text{_OTHER_}, & \text{otherwise.} \end{cases} \quad (11)$$

Missing is treated as an explicit level before the frequency cutoff. One-hot coordinates are then $I[g_j(c) = v]$ for $v \in \mathcal{V}_j \cup \{\text{_OTHER_}\}$. This bounded encoding prevents a neighborhood or ZIP variable from creating thousands of singleton dimensions. More importantly, unseen test categories have a defined representation rather than causing schema failure.

3.5.3 Deterministic 311 sampling

Let u_i be the unique request key and $h(u_i)$ a deterministic unsigned 64-bit hash. For sampling rate $p = 0.01$, row i is selected when

$$h(u_i) \bmod 10,000 < 10,000p. \quad (12)$$

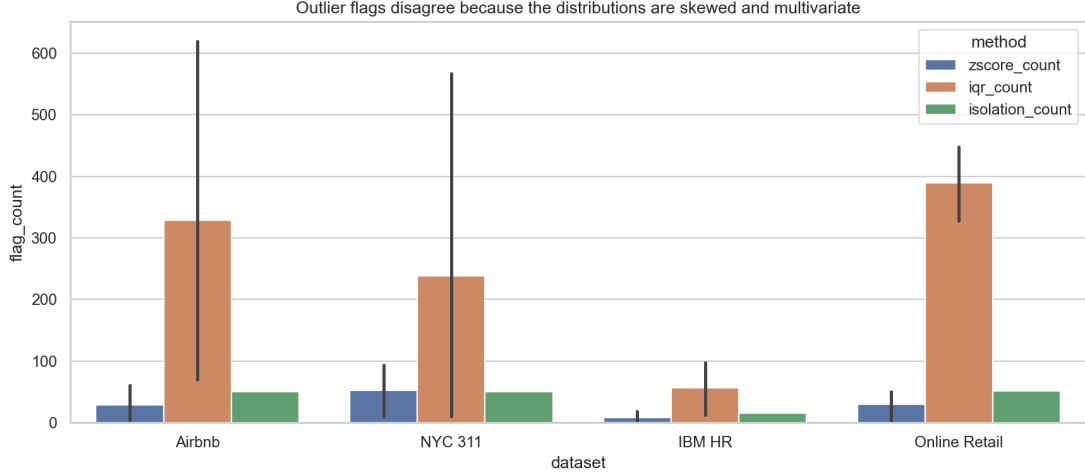


Figure 1: Counts flagged by Z-score, IQR, and multivariate Isolation Forest. Disagreement is expected because each method defines anomaly differently.

The inclusion decision depends only on the stable key. It is invariant to chunk size and file order, unlike taking the first n rows or calling a stateful random generator independently inside each chunk. The exact population marginals accumulated during the same pass make the sample auditable rather than merely convenient.

The cached filename includes source byte size, modification timestamp, and rate. This creates a simple data-lineage rule: if the source changes, the cache key changes and a fresh pass is forced. Caching is computational engineering, but it also protects reproducibility by preventing different notebook runs from silently drawing different records.

3.5.4 Duplicate semantics

Exact duplicate rows and repeated unique request keys are genuine storage duplicates and are removed. Near-duplicates are different. Define

$$q_i = (\text{complaint}, \text{detail}, \text{round}(\text{lat}, 4), \text{round}(\text{lon}, 4), \lfloor t/10 \text{ min} \rfloor). \quad (13)$$

The feature

$$c_i = \sum_j I(q_j = q_i) \quad (14)$$

counts reports in the same complaint/location/time cell. Those rows are retained because independent callers are evidence of intensity. Blind deduplication would impose the untested assumption that only one report per incident is valid.

3.5.5 Resolution-time target

For a closed request,

$$y_i = \frac{t_i^{\text{close}} - t_i^{\text{create}}}{3600} \quad (15)$$

in hours. Negative durations are invalid. Three forms are retained:

$$y_i^{\text{raw}} = y_i, \quad (16)$$

$$y_i^{\text{cap}} = \min(y_i, Q_{0.995}), \quad (17)$$

$$y_i^{\text{log}} = \log(1 + y_i). \quad (18)$$

Models use the log target and are evaluated after inverse transformation on raw hours. The capped target is a robustness diagnostic. Open requests have no t_i^{close} and are excluded from ordinary regression rather than assigned a value.

3.5.6 Airbnb target variants

Airbnb price uses the same raw/log distinction. The winsorized sensitivity target is group-specific. For group $g = (\text{city}, \text{room type})$, define lower and upper bounds

$$L_g = Q_{0.005}(y \mid g), \quad U_g = Q_{0.995}(y \mid g). \quad (19)$$

Then

$$y_i^{\text{win}} = \min\{\max(y_i, L_g), U_g\}. \quad (20)$$

Groups with fewer than 30 nonmissing observations use global bounds. This is preferable to one global threshold because \$500 can be ordinary for one market/room type and extreme for another. The report's primary metrics still use observed positive price.

3.5.7 HR association and selection

For numerical candidate driver X_j , the standardized mean difference is

$$d_j = \frac{\bar{x}_{j, \text{attrition}} - \bar{x}_{j, \text{stay}}}{\sqrt{(s_{j, \text{attrition}}^2 + s_{j, \text{stay}}^2)/2}}. \quad (21)$$

The largest magnitudes occur for job level (-0.493), total working years (-0.484), monthly income (-0.479), years in role (-0.462), years with manager (-0.449), and age (-0.425). Negative values mean lower levels are associated with attrition. Categorical drivers are summarized by the range of group attrition rates: job role has the largest spread (0.373), followed by overtime (0.201), travel frequency (0.169), and marital status (0.154).

Mutual information is estimated from training data only:

$$I(X; Y) = \sum_{x, y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}. \quad (22)$$

Numerical features are rank-quantile discretized for this filter estimate. Overtime, job role, monthly income, total experience, and job level rank highly. Mutual information is a nonlinear dependence score, not a causal coefficient, and correlated predictors can share or duplicate information.

3.5.8 Retail customer aggregation

For customer c and historical cutoff T , recency is

$$R_c = T - \max\{t_i : i \in c, t_i < T\}, \quad (23)$$

frequency is the number of distinct positive-purchase invoices, and monetary value is

$$M_c = \sum_{i \in c, t_i < T} q_i p_i I(q_i > 0, \text{not cancellation}). \quad (24)$$

Return value is tracked separately from purchases. Return rate is

$$\rho_c = \frac{\text{return value}_c}{M_c + \text{return value}_c}. \quad (25)$$

Basket features are first computed at invoice level and then averaged at customer level so a 50-line order is not mistaken for 50 independent visits.

Let T_0 be the beginning of observation and $T_m = (T_0 + T)/2$. Spending trend is

$$\tau_c = \frac{1 + M_{c, [T_m, T]}}{1 + M_{c, [T_0, T_m]}}. \quad (26)$$

The additive one avoids division by zero. A large τ_c identifies accelerating customers; a very small value identifies declining activity. The \$57,100 worst-case customer has only \$21.84 of recent historical spend and therefore demonstrates that a trend feature cannot predict a genuinely abrupt regime change.

3.6 Label quality

Labels are measurements produced by institutions, not natural constants. Recording their provenance, intended use, and known limitations is part of responsible dataset documentation [?].

Airbnb. There are 78 prices at or above \$10,000. The most extreme worst-case listing is a Seattle hotel room labeled \$50,000 with no reviews and predicted at \$131.56. The model failure may be a legitimate tail failure, a blocked-calendar sentinel, or a scrape artifact. Superhost is a platform-defined status that changes over time. A host can resemble a superhost in review and response features while failing an unobserved eligibility rule.

NYC 311. Complaint labels are selected or normalized through a taxonomy. “Street Condition,” “Traffic Signal Condition,” “Blocked Driveway,” and “Illegal Parking” overlap in location and agency. The **OTHER** class groups heterogeneous rare categories, so it is intentionally hard to model. Resolution closure can be administrative, and reopened cases make one closure timestamp an imperfect service-quality endpoint.

HR. Attrition says that an employee left, not why. Voluntary resignation, retirement, dismissal, relocation, and contract completion may be combined. The dataset’s synthetic/demonstration character limits transportability.

Retail. Future segments are constructed labels. Inactivity during a 100-day future window can mean churn, seasonality, purchase through another identity, or simply no need to repurchase. Tertile boundaries create adjacent customers with nearly identical spend but different labels.

4 Experimental Setup and Model Development

4.1 Splits and preprocessing

Airbnb uses host-group-disjoint splitting. NYC uses chronological holdout, because a random split would mix earlier and later operational regimes. HR uses the fixed stratified 75/25 split, preserving the real 16% prevalence in the untouched test set. Retail targets are already future-defined; model rows are then seeded-split for the controlled algorithm comparison.

Exact kernel methods are evaluated on the controlled sample sizes in ???. This is required for comparability: every required model sees the same rows, while the computational analysis reports why the full Gram matrix is infeasible.

The preprocessor is a custom class:

1. identify numerical and categorical columns from training data;
2. replace numerical gaps with the training median;
3. robust-scale with training median and IQR,

$$x' = \frac{x - \text{median}(x)}{Q_3 - Q_1};$$

4. if IQR is zero, use standard deviation, then 1 as a final guard;
5. retain at most 18–30 frequent categorical levels, depending on task;
6. map unseen or rare levels to `__OTHER__`;
7. one-hot encode using the learned level order.

This preprocessing is particularly important for KNN and RBF kernels. Without scaling, a variable measured in thousands, such as monthly income, dominates a satisfaction score measured from 1 to 4.

4.2 Metrics

For classification, accuracy is reported but model selection uses macro-F1:

$$F1_k = \frac{2P_k R_k}{P_k + R_k}, \quad \text{MacroF1} = \frac{1}{K} \sum_{k=1}^K F1_k. \quad (27)$$

Balanced accuracy is mean per-class recall. These metrics prevent the largest class from defining success. This is essential for HR and 311 [?].

For regression:

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad (28)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \quad (29)$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (30)$$

MAE selects the model because it is less dominated by extreme tails. RMSE and R^2 expose tail failure.

4.3 From-scratch non-kernel methods

4.3.1 Logistic regression

For K classes, weights $W \in \mathbb{R}^{(d+1) \times K}$ produce logits $z_i = \tilde{x}_i^\top W$, where $\tilde{x}$ includes an intercept. Softmax probabilities are

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_\ell \exp(z_{i\ell})}. \quad (31)$$

The loss is

$$\mathcal{L}(W) = -\frac{1}{n} \sum_i \sum_k y_{ik} \log p_{ik} + \frac{\lambda}{2} \|W_{-0}\|_F^2. \quad (32)$$

The implementation uses stable `logsumexp`, excludes the intercept from regularization, and optimizes with Adam. Every ten epochs it evaluates full loss and stops if the improvement falls below tolerance.

Conceptually, logistic regression asks whether the log odds are approximately additive in the encoded features. It is strong when the feature engineering has already exposed the important ratios and categories and when sample size is insufficient to estimate a highly curved boundary.

4.3.2 Linear regression

The implementation solves ridge-stabilized normal equations:

$$\hat{\beta} = (\tilde{X}^\top \tilde{X} + \lambda D)^{-1} \tilde{X}^\top y, \quad (33)$$

where $D_{00} = 0$ so the intercept is not penalized. If the solve is singular, the Moore–Penrose pseudoinverse is used. Regression is trained on $\log(1+y)$ for price, duration, and spend:

$$\tilde{y} = \log(1+y), \quad \hat{y} = \max\{0, \exp(\hat{\tilde{y}}) - 1\}. \quad (34)$$

A training-only 0.1–99.9 percentile guard clips extreme transformed predictions before exponentiation. The clipped fraction is reported; it is zero for most models and at most 2.33% in these experiments.

4.3.3 K-nearest neighbors

Euclidean distances are computed in batches using SciPy:

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2. \quad (35)$$

The $k = 9$ nearest points are selected by partial partition rather than full sort. Distance weights are

$$w_i = \frac{1}{\max(d_i, 10^{-9})}. \quad (36)$$

Regression returns the weighted mean. Classification returns the class with maximum weighted vote.

KNN has zero parametric training cost but expensive prediction and high sensitivity to scale, irrelevant dimensions, and sparse one-hot geometry [?]. It is a useful diagnostic: poor KNN performance often indicates that local Euclidean neighborhoods do not align with labels.

4.3.4 Decision tree

The scratch CART implementation [?] searches quantile candidate thresholds. Classification impurity is Gini:

$$G(S) = 1 - \sum_k p_k^2. \quad (37)$$

Regression impurity is variance. A split maximizes

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R). \quad (38)$$

Depth is limited to 7, nodes require at least 24 samples to split and 8 per leaf, and at most 45 features and 20 thresholds per feature are considered. These controls are computational regularization as well as statistical regularization.

4.4 Kernel methods

4.4.1 Kernel functions

The kernel trick replaces explicit features $\phi(\mathbf{x})$ with inner products

$$K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle. \quad (39)$$

Three kernels are compared:

$$K_{\text{lin}}(\mathbf{x},) = \mathbf{x}^\top, \quad (40)$$

$$K_{\text{poly}}(\mathbf{x},) = (\gamma \mathbf{x}^\top + 1)^2, \quad (41)$$

$$K_{\text{RBF}}(\mathbf{x},) = \exp(-\gamma \|\mathbf{x} - \cdot\|_2^2). \quad (42)$$

The RBF/polynomial scale uses the median squared-distance heuristic:

$$\gamma = \frac{1}{\text{median}_{i < j} \|\mathbf{x}_i - \mathbf{x}_j\|_2^2}. \quad (43)$$

4.4.2 Kernel SVM

The classical SVM is a maximum-margin classifier [?]. The implementation uses the representer form

$$f_k(\mathbf{x}) = \sum_{j=1}^n \alpha_{jk} K(\mathbf{x}, \mathbf{x}_j) + b_k \quad (44)$$

and one-vs-rest regularized squared hinge loss:

$$\mathcal{L}(\alpha, b) = \frac{\lambda}{2} \sum_k \alpha_k^\top K \alpha_k + \frac{1}{n} \sum_{i,k} \max(0, 1 - y_{ik} f_k(\mathbf{x}_i))^2. \quad (45)$$

Here $y_{ik} \in \{-1, +1\}$. Adam updates α and b . Squared hinge gives a continuous gradient away from the hinge and is straightforward to implement without a quadratic-programming library.

4.4.3 Kernel ridge regression

Kernel ridge regression combines ridge regularization [?] with the representer theorem:

$$\hat{f}(\mathbf{x}) = \sum_i \alpha_i K(\mathbf{x}, \mathbf{x}_i) + \bar{y}, \quad \alpha = (K + \lambda I)^{-1}(y - \bar{y}). \quad (46)$$

It is smooth, deterministic, and exposes the kernel matrix cost directly.

4.4.4 Kernel KNN

Distances in feature space are obtained without constructing ϕ :

$$\|\phi(\mathbf{x}) - \phi(\cdot)\|^2 = K(\mathbf{x}, \mathbf{x}) + K(\cdot, \cdot) - 2K(\mathbf{x}, \cdot). \quad (47)$$

An important theoretical limitation appears for the stationary RBF kernel:

$$d_\phi^2 = 2 - 2 \exp(-\gamma \|\mathbf{x} - \cdot\|^2). \quad (48)$$

This is monotonic in Euclidean distance, so neighbor ordering is unchanged. RBF kernel KNN can alter weighting but cannot create new nearest neighbors. The near-identical ordinary KNN and RBF-KNN results are therefore expected, not accidental.

4.4.5 Kernel PCA plus logistic regression

KPCA centers the Gram matrix:

$$K_c = H K H, \quad H = I - \frac{1}{n} \mathbf{1} \mathbf{1}^\top, \quad (49)$$

then solves

$$K_c v_j = \lambda_j v_j. \quad (50)$$

The top 12 normalized nonlinear components are sent to the scratch logistic classifier [?]. KPCA is unsupervised: it preserves directions of kernel variance, not necessarily directions useful for the label. This explains why high-variance RBF components can perform poorly under imbalance.

4.5 Imbalance interventions

HR compares three training distributions:

- original: 1,103 rows at real prevalence;
- undersampled: 178 attrition and 178 non-attrition rows;
- mixed SMOTE: 925 rows per class.

The mixed-type SMOTE implementation follows the interpolation idea of [?]. Numerical minority features are robust-scaled for neighbor search and interpolated:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + u(\mathbf{x}_j - \mathbf{x}_i), \quad u \sim U(0, 1). \quad (51)$$

Each categorical feature is copied from either parent. Only training data are synthesized.

5 Results

5.1 Classification

Airbnb superhost. Logistic regression is best with accuracy 0.770, balanced accuracy 0.762, and macro-F1 0.764. The polynomial SVM reaches 0.730 macro-F1 and the decision tree 0.728. The result indicates that, after review, response, host-tenure, property, and city engineering, the remaining decision is approximately additive. The host-group split is important: it prevents host-specific repetition from making local methods look artificially strong.

NYC complaint. The linear-kernel SVM is best in macro-F1 (0.274) and balanced accuracy (0.329), although logistic regression has higher raw accuracy (0.562 versus 0.519). The SVM sacrifices majority-class accuracy to recover more minority classes. The distinction is operationally important: a classifier that predicts common illegal parking and noise complaints can have acceptable accuracy while failing rare service categories.

HR attrition. Logistic regression has the highest macro-F1 (0.681) and accuracy (0.866). The linear-kernel SVM has lower macro-F1 (0.662) but higher balanced

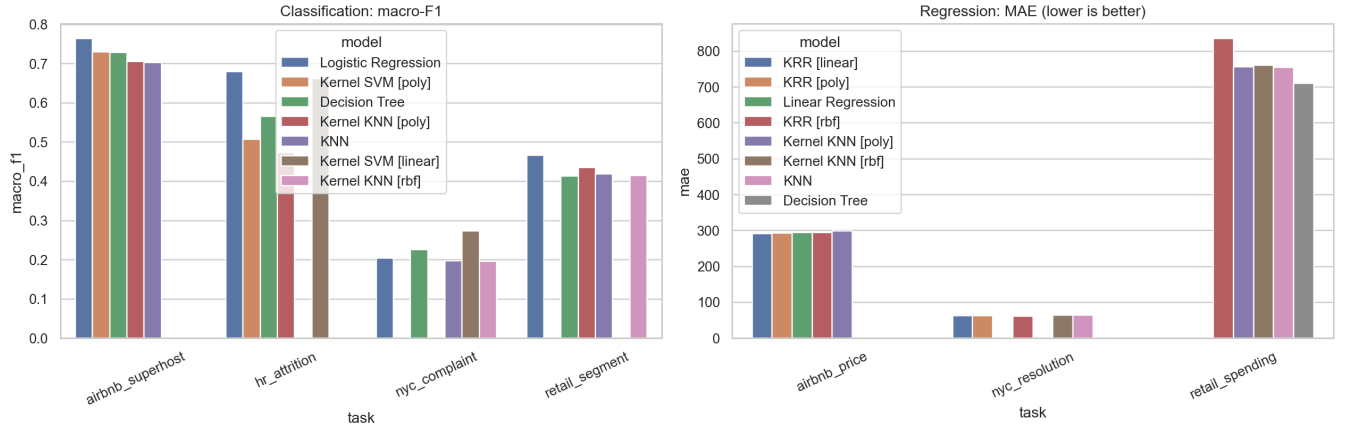


Figure 2: Overview of the best five models per task. Classification uses macro-F1; regression uses MAE.

Table 2: Classification macro-F1. Bold indicates the best result per task.

Model	Airbnb superhost	HR attrition	NYC complaint	Retail segment
Logistic regression	0.764	0.681	0.204	0.466
KNN	0.703	0.473	0.197	0.419
Decision tree	0.728	0.565	0.225	0.414
Kernel SVM, linear	0.642	0.662	0.274	0.339
Kernel SVM, polynomial	0.730	0.507	0.089	0.399
Kernel SVM, RBF	0.675	0.456	0.120	0.408
Kernel KNN, polynomial	0.706	0.474	0.192	0.436
Kernel KNN, RBF	0.703	0.473	0.196	0.415
RBF KPCA + logistic	0.675	0.456	0.109	0.358

accuracy (0.704 versus 0.646), indicating more aggressive minority detection with more majority errors. RBF SVM, RBF KPCA, and RBF KNN are close to majority prediction, with balanced accuracy near 0.5.

SMOTE changes the tradeoff. Logistic regression trained on mixed SMOTE reaches balanced accuracy 0.771 and macro-F1 0.681, nearly identical macro-F1 to original logistic but lower accuracy (0.766). Undersampling reaches balanced accuracy 0.735 but macro-F1 0.636 because discarding 747 majority examples harms precision. Thus, resampling is not “free accuracy”; it chooses a different error cost.

Retail segment. Logistic regression is best with macro-F1 0.466. Polynomial kernel KNN reaches 0.436. The four labels overlap because low/mid/high are discretized future spend and inactivity is an observation-window state. The weak absolute performance is evidence that a short historical profile does not uniquely determine a customer’s next-period tier.

5.2 Regression

Airbnb price. Linear KRR improves MAE from 293.97 to 291.97 dollars, only 0.68%. RMSE remains about \$2,935 and R^2 is slightly negative because the test set

contains extreme prices. The correct interpretation is not “KRR wins.” It is that all smooth models fit ordinary listings similarly and fail the rare target tail. The negative R^2 is dominated by a \$50,000 listing and demonstrates why MAE and failure analysis must accompany a single aggregate score.

NYC resolution. RBF KRR is best with MAE 60.90 hours, improving 5.19% over KNN. Polynomial KRR is second at 62.16. RMSE remains about 170 hours and R^2 about 0.066, meaning that extreme multi-week resolutions are poorly explained. The RBF gain is nevertheless meaningful: agency, channel, time, and geography form localized operational regimes for the bulk of cases.

Retail spending. The decision tree is best with MAE \$710.30 and $R^2 = 0.187$. KNN is second at \$754.80. Every kernel smoother is worse. The point mass at zero, threshold behavior, country/customer heterogeneity, and rare extreme spenders favor partitioning over one global smooth surface. Linear and KRR models also suffer from exponential back-transformation after log-target fitting.

Table 3: Regression MAE on original target units. Bold indicates the best result.

Model	Airbnb price (\$)	NYC resolution (hours)	Retail spending (\$)
Linear regression	293.97	65.65	880.17
KNN	299.60	64.24	754.80
Decision tree	303.03	77.06	710.30
KRR, linear	291.97	63.48	856.74
KRR, polynomial	292.47	62.16	866.85
KRR, RBF	293.98	60.90	834.74
Kernel KNN, polynomial	299.23	65.31	756.08
Kernel KNN, RBF	299.72	64.20	760.48

6 Kernel Investigation

6.1 Geometry diagnostics

Predictive metrics alone do not explain a kernel. Three diagnostics are used.

Centered kernel alignment. For centered kernel K_c and centered target kernel Y_c :

$$A(K, Y) = \frac{\langle K_c, Y_c \rangle_F}{\|K_c\|_F \|Y_c\|_F}. \quad (52)$$

For classification, $Y_{ij} = 1$ when labels match. For regression, $Y = (y - \bar{y})(y - \bar{y})^\top$. Alignment measures whether pairwise similarity agrees with target similarity.

Effective rank. With nonnegative eigenvalues λ_i :

$$r_{\text{eff}} = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2}. \quad (53)$$

A very low rank means most similarity variation lies in a few directions. A very high rank on a small sample can indicate a nearly identity kernel with poor sharing.

Off-diagonal concentration. The mean and standard deviation of $K_{ij}, i \neq j$ indicate whether points become almost identical or almost orthogonal. An RBF scale that makes all off-diagonal values nearly zero turns interpolation into memorization.

6.2 Why alignment did not always predict performance

RBF has the highest alignment for six of seven tasks, yet RBF models are often not best. Alignment is an in-sample pairwise statistic; it does not incorporate estimator regularization, class imbalance, sample size, or the downstream objective. An aligned kernel with effective rank around four may compress rare-class distinctions. KPCA compounds the problem by selecting variance directions without labels.

6.3 Dataset-specific conclusions

NYC benefited most. NYC has the largest average normalized kernel benefit, approximately 0.050. Resolution time benefits from RBF KRR because geographically and operationally nearby requests can share local duration behavior. Complaint classification benefits from a kernel SVM, but the best kernel is linear rather than RBF. This is not contradictory. One-hot agency, borough, and channel variables plus cyclic time features already create a useful expanded representation; a margin objective improves minority separation, while additional RBF curvature collapses classes.

Airbnb benefited little. The average kernel benefit is slightly negative (-0.0135). Linear KRR improves price MAE by only 0.68%; logistic regression is clearly best for superhost. Host-group splitting removes a source of artificial local similarity. The remaining rare price tail cannot be learned by smooth kernels from a 700-row sample.

HR benefited least among classification domains. The average benefit is -0.0182 . The dataset is small and low-cardinality. Logistic regression’s additive regularized boundary has lower variance. Linear SVM is useful if balanced recall is prioritized, but nonlinear RBF and polynomial geometry mostly predict the majority.

Retail benefited least overall. Average kernel benefit is -0.0474 . Future spending is zero-inflated and discontinuous; segments are constructed and overlapping. The decision tree directly isolates threshold regions, while kernel smoothers average inactive, ordinary, and extreme customers.

7 Error and Failure Analysis

The ten worst predictions are not treated as embarrassing exceptions. They are evidence about what the feature representation and target cannot support.

Table 4: Kernel alignment and effective rank on controlled training subsets.

Task	Kernel	Alignment	Eff. rank	Task	Kernel	Alignment	Eff. rank
Airbnb price	Linear	0.080	5.93	Airbnb superhost	Linear	0.052	6.07
	Polynomial	0.042	6.51		Polynomial	0.044	6.83
	RBF	0.141	4.61		RBF	0.086	4.14
NYC complaint	Linear	0.185	11.84	NYC resolution	Linear	0.135	11.88
	Polynomial	0.191	4.02		Polynomial	0.042	4.78
	RBF	0.209	6.05		RBF	0.155	6.15
HR attrition	Linear	0.057	11.97	Retail segment	Linear	0.129	3.36
	Polynomial	0.055	4.49		Polynomial	0.045	2.50
	RBF	0.071	6.48		RBF	0.143	4.07
Retail spending	Linear	0.131	3.26				
	Polynomial	0.064	2.31				
	RBF	0.104	3.64				

7.1 Airbnb price

The largest error is the \$50,000 Seattle hotel room predicted at \$131.56. The next is a \$9,400 six-bedroom Seattle property predicted at \$406.99. Other severe underpredictions occur at \$2,607, \$2,283, \$1,712, and \$1,340. Nine of the ten worst cases are large underpredictions; one \$200 listing is overpredicted at \$866.

The systemic limitation is regression to the center after log transformation and regularization. Extreme prices are rare in the 700-row training subset, and a smooth model has no reason to extrapolate by two orders of magnitude. The possible data artifact is that \$50,000 may encode a platform sentinel or unavailable calendar. The correct conclusion requires both: the model lacks tail support, and the label itself deserves validation.

7.2 Airbnb superhost

Among the ten displayed errors, eight non-superhosts are predicted as superhosts and two superhosts as non-superhosts. The false positives often have high response rates, strong reviews, and professional listing behavior. This suggests an omitted-rule problem: superhost status depends on platform eligibility and time windows not completely represented by the snapshot. The false negatives show that recent or sparse review history can hide an actual status.

These are primarily ambiguous-ground-truth and omitted-feature failures rather than evidence that logistic regression cannot form a nonlinear boundary.

7.3 NYC complaint

Errors include Street Condition predicted as Traffic Signal Condition, Noise-Street/Sidewalk predicted as Illegal Parking, OTHER predicted as Street Condition or Water System, and Abandoned Vehicle predicted as HEAT/HOT WATER or Illegal Parking. These cases expose three issues:

1. **Taxonomy overlap:** several categories share agency, street location, borough, and channel.
2. **Information removed with the target:** the raw complaint text cannot be used because it directly defines the label; remaining metadata may be insufficient.
3. **Rare-class geometry:** macro-F1 remains low because minority categories have few controlled-subset examples.

The failure is both systemic and label-related. A production solution would use non-leaking free-text description available at intake, hierarchical labels, or cost-sensitive learning.

7.4 NYC resolution

The worst actual durations are 1,440, 1,241, 1,141, and 1,149 hours, predicted between 28 and 97 hours. The leading case is a Manhattan restaurant complaint submitted by phone on a weekend. Other cases are residential-building complaints handled by HPD.

The model learns ordinary closure behavior but not long administrative or investigative delays. Relevant unobserved variables include agency backlog, inspection scheduling, case severity, staffing, weather, holidays, and reopening. Closed-case-only training also creates selection bias: the model estimates observed completion time, not the eventual duration of all requests.

7.5 HR attrition

Eight of the ten selected errors are attrition cases predicted as stayers; two are stayers predicted as leavers. False negatives include employees with apparently stable profiles: no overtime, long tenure, reasonable satisfaction, and established roles. Attrition can be triggered by external offers, personal events, conflict, or restructuring—variables absent from the dataset. One false negative has overtime, single status, low income, and promotion stagnation, illustrating that even recognizable risk factors

do not deterministically imply departure.

The dominant problem is irreducible overlap and omitted causes. Resampling can increase recall, but it cannot create information that was never measured.

7.6 Retail segment

Five of the ten worst segment cases are **Future_Low** predicted inactive. Two mid-tier cases are predicted inactive, and one high-tier customer is predicted inactive. Many have a single historical order and long recency. Their histories legitimately resemble churn, yet they purchase later. The model failure is uncertainty from sparse histories, while the label failure is that a finite future window turns random purchase timing into a hard category.

7.7 Retail spending

The largest failure is extraordinary: a customer with only \$21.84 observed historical spend has \$57,100.67 future spend, but the tree predicts \$6.92. This behavior is impossible to infer from the historical features and may represent a wholesale transition, identity mismatch, or one-off event. Other severe errors involve actual future spend of \$25,332, \$22,921, and \$18,005. The tree assigns several of them the same \$9,346 leaf prediction.

This is a systemic extrapolation limit. Trees cannot predict beyond leaf averages; KNN cannot exceed local target neighborhoods; smooth kernels shrink tails. A two-stage model—probability of activity followed by conditional heavy-tail regression—or a Tweedie/quantile objective would better match the target distribution, but no method can foresee an abrupt customer regime change without explanatory signals.

8 Cross-Dataset Comparison

8.1 Hardest to clean

NYC 311 is the hardest to clean. It combines 3.4 million rows, a 2.8 GB file, duplicates and repeated incidents, missing coordinates, evolving categories, temporal drift, channel bias, and censored outcomes. The engineering pipeline must solve computational access and statistical validity simultaneously.

Airbnb has more raw missingness—one field is entirely empty and several exceed 70%—but its table fits in memory and listing identity is clear. Retail is large but structurally regular once returns and customer aggregation are defined. HR is the easiest to clean but the hardest to trust externally.

8.2 Most aggressive feature engineering

Online Retail requires the most aggressive feature engineering because the raw and modeled units differ. Hun-

dreds of thousands of lines become 3,317 customers. The target is not present; it is created by a future cutoff. RFM, basket, return, timing, diversity, and trend features summarize a stochastic sequence.

NYC requires substantial temporal and operational engineering, but its row already represents a request. Airbnb requires parsing and domain features without changing the observational unit. HR requires only ratios and selection.

8.3 Most and least kernel benefit

NYC benefits most because local operational regimes exist in space, agency, channel, and time. Retail benefits least because zero inflation and regime changes violate smooth similarity. HR also benefits little because sample size is insufficient for flexible curvature. Airbnb’s host-group split reveals that apparent local similarity is not necessarily transferable.

8.4 A general lesson about manifolds

The word “manifold” should not be interpreted as a guarantee of a smooth low-dimensional surface. In real data:

- some dimensions are categories represented by one-hot vertices;
- some points are censored outcomes;
- some labels are institutional taxonomies;
- some targets contain point masses and heavy tails;
- some observations are dependent through host, customer, place, or time.

The practical manifold is the geometry induced by measurement, preprocessing, and the kernel. A mathematically valid kernel can be operationally wrong if its notion of similarity ignores those structures.

9 Discussion: Bias, Computation, and Theory Versus Reality

9.1 Bias and fairness

9.1.1 Airbnb

Geographic representation is uneven: Nashville contributes 39.8% of rows. City-specific regulations, tourism demand, housing stock, and scrape dates are confounded with city labels. Price MAE varies sharply: Bozeman \$24, Nashville \$89, Dallas \$136, and Seattle \$650. Seattle’s result is driven by extreme hotel prices and shows that global metrics hide regional unreliability.

Superhost macro-F1 ranges from 0.629 in small-sample Bozeman to 0.818 in Seattle. This is not a protected-class fairness audit because demographic attributes are absent. It is a geographic reliability audit. Deployment claims must be limited to represented cities and scrape periods.

9.1.2 NYC 311

311 data measure reporting and service access as well as underlying conditions. Smartphone/internet access, trust in government, language, knowledge of 311, and willingness to report differ across communities. Complaint classifier macro-F1 is 0.158 in Queens, 0.239 in the Bronx, 0.346 in Brooklyn, 0.362 in Manhattan, and 0.417 in Staten Island, although small group sizes make the last estimate uncertain.

Channel matters: complaint macro-F1 is 0.297 online but 0.208 by phone. Resolution MAE is 15.6 hours for mobile, 53.1 online, and 113.5 by phone. These differences can reflect complaint mix and agency routing, not service discrimination alone. A causal fairness claim would require adjustment for complaint severity and agency. Nevertheless, unequal error means reliability differs by access pathway.

Temporal drift is visible: complaint macro-F1 falls from 0.312 in April and 0.320 in May to 0.159 in June. Resolution MAE falls to 21.5 hours in June but this may be censoring: recent long cases have not yet closed and are absent from regression. This is a direct example of apparently improved performance caused by outcome availability.

9.1.3 HR

The model has accuracy 0.843 for women and 0.881 for men, while balanced accuracy is 0.666 and 0.629 respectively. Sales has much stronger macro-F1 (0.825) than R&D (0.595) or HR (0.467), partly because HR has only 16 test cases. Overtime employees have lower accuracy but higher balanced accuracy, reflecting a different class mix.

Gender is available, but the dataset omits race, disability, age-related legal context beyond numerical age, caregiving, job market, location, and organizational hierarchy. A model cannot be certified fair with respect to unmeasured groups. Worse, using attrition predictions for employment decisions can create feedback loops: a worker labeled high-risk may receive fewer opportunities, causing departure.

9.1.4 Retail

The United Kingdom represents 88.8% of retained transactions and 266 of 300 segment test customers. Germany has only eight test customers and segment macro-F1 0.321. Country-level R^2 is unstable, including negative values for France and Spain. Anonymous customers are excluded, so results apply to identified repeat customers rather than all transactions.

9.2 Computational analysis

For n examples, a dense float64 Gram matrix requires

$$M_K = 8n^2 \text{ bytes.} \quad (54)$$

KRR solves a dense linear system and KPCA eigendecomposes a dense matrix, both $O(n^3)$ in the direct implementation. Full engineered sizes imply:

- Airbnb price: 2.77 GB Gram matrix;
- Airbnb superhost: 3.92 GB;
- NYC complaint: 8.76 GB;
- NYC resolution: 7.99 GB;
- HR: 0.016 GB;
- Retail: 0.082 GB.

These values describe one matrix. Cross-validation, simultaneous kernels, centered copies, factorization work arrays, and target matrices multiply memory. NYC complaint has an n^3 operation proxy of 4.03×10^{13} , compared with 7.41×10^7 on the 420-row experiment.

Measured controlled-subset fit times are small because n is deliberately bounded. Linear regression fits in under a millisecond. KRR takes roughly 0.02–0.03 seconds at $n = 650$ –700. KPCA and kernel SVM allocate tens of MB. The scratch decision tree is often slowest (0.3–1.2 seconds) because Python loops search features and thresholds, whereas linear algebra runs in optimized native code.

Production-scale kernels require approximation or structure: Nyström landmarks [?], random Fourier features [?], iterative solvers, sparse support vectors, or local kernels. The computational limitation is not an implementation inconvenience; it changes which hypothesis classes are available on commodity hardware.

9.3 Theory versus reality

Textbook datasets overestimate performance for five reasons.

The observational unit is pre-solved. Iris has one flower per row. MNIST has one image per row. Retail starts with line items and requires a temporal customer definition.

The target is treated as unambiguous. Textbook classes are stable. 311 categories overlap, superhost follows platform rules, attrition merges departure causes, and future retail tiers are constructed.

Missingness is minimized or already encoded. Real missingness contains structural states and censoring. A blank closure date is not the same type of absence as a blank coordinate.

Train and test are assumed exchangeable. Random splits ignore host dependence, customer identity, temporal

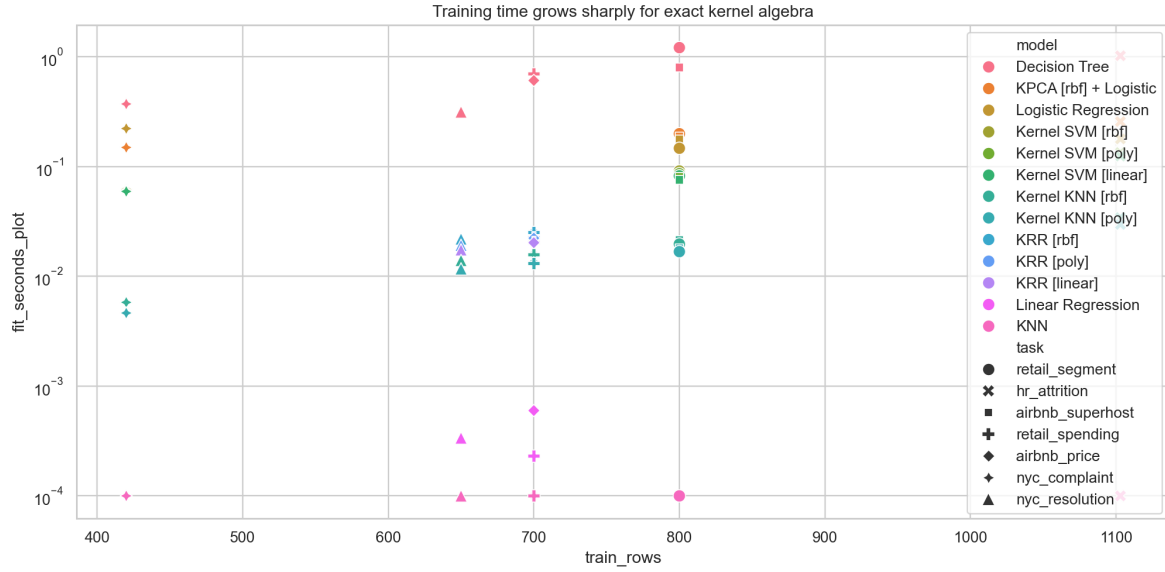


Figure 3: Measured fit time for controlled subsets. Python memory profiling undercounts native NumPy/BLAS buffers, so analytical Gram-matrix sizes are also reported.

drift, and future leakage. Once Airbnb is split by host and 311 by time, performance becomes more realistic.

Computational complexity is hidden by small n . An RBF kernel is a one-line formula on Iris. On 34,283 requests, one matrix is 8.76 GB and direct cubic algebra is prohibitive.

The empirical evidence is direct. Accuracy and macro-F1 diverge on 311 and HR. Worst errors cluster in target tails and sparse histories. Group metrics vary by city, borough, channel, department, and country. Kernel alignment does not guarantee generalization. Data engineering determines whether the experiment answers a meaningful question. This is the technical debt principle of machine learning: model code is a small fraction of system complexity [?].

10 Conclusion

This study implemented and evaluated linear, distance-based, tree, and kernel methods across four imperfect real-world datasets. The main result is not a universal ranking. It is a disciplined map from data structure to model behavior.

NYC 311 was hardest to clean and benefited most from kernels, specifically RBF KRR for localized resolution regimes and a margin-based linear kernel for complaint imbalance. Online Retail required the most aggressive feature engineering and benefited least from exact kernels because the target is zero-inflated, heavy-tailed, and temporally unstable. Airbnb showed that group-disjoint evaluation and target tails can eliminate apparent nonlinear gains. IBM HR showed that a simple logistic model

can outperform nonlinear kernels when data are small and low-cardinality, while resampling changes recall/precision tradeoffs rather than creating information.

The assignment’s central objective is therefore achieved: the success or failure of an algorithm is inseparable from the dataset definition. A strong practitioner must be able to derive the observational unit, defend the missingness assumption, distinguish anomaly from error, prevent leakage, respect time, analyze label provenance, choose metrics that reflect imbalance, understand kernel geometry, and explain failures at the row level. Only after those tasks are complete does “which algorithm is best?” become a scientifically meaningful question.

A Implementation Guide to the Python Code

The executable deliverables are:

- `ML_Homework_Complete.ipynb`: orchestration, narrative, data engineering, diagnostics, experiments, and artifact generation;
- `src/scratch_ml.py`: all core estimators, preprocessing, metrics, quality tests, and kernel diagnostics;
- `src/homework_experiments.py`: task preparation, split logic, model registry, timing, prediction storage, group metrics, and worst-case extraction;
- `processed/model_tables/`: engineered task tables;
- `artifacts/`: CSV results and figures used by this report;
- `cache/`: source-versioned 311 sample and population marginals.

A.1 How to read `scratch_ml.py`

Splitting utilities. `stratified_split_indices` loops over classes, shuffles class positions with a seeded NumPy generator, and allocates each class proportionally. `stratified_subsample_indices` computes an integer class allocation for controlled kernel subsets. These functions return positions rather than copying data.

TabularPreprocessor. `fit` stores input column order, numerical medians, robust centers/scales, and bounded categorical vocabularies. `transform` reindexes to the training schema, applies stored numerical values, maps rare/unseen categories to `__OTHER__`, and returns a float64 matrix. The class intentionally resembles a production fit/transform contract: no test statistic is learned.

LogisticRegressionScratch. The class constructs an intercept column, one-hot target matrix, and zero weight matrix. Each epoch computes stable softmax probabilities, gradient of cross-entropy plus L_2 , and Adam moments. `predict_proba` repeats stable softmax; `predict` selects the largest probability.

LinearRegressionScratch. The class builds $\tilde{X}^\top \tilde{X} + \lambda D$ and solves it directly. The fallback pseudoinverse protects against singular one-hot combinations.

KNNScratch. Training stores arrays. Prediction processes test batches, calls `scipy.spatial.distance.cdist`, uses `argpartition` for k neighbors, and applies inverse-distance weighting. This avoids allocating the full test-by-train distance matrix when test data are large.

DecisionTreeScratch. Each node stores prediction, feature, threshold, left, and right. `_grow` stops by depth, node size, or zero impurity. It samples a feature subset, proposes unique midpoints or quantiles, computes gain, and recurses. The implementation is transparent but slower than vectorized compiled CART libraries.

Kernel functions. `pairwise_sq_dists` uses

$$\|x - z\|^2 = \|x\|^2 + \|z\|^2 - 2x^\top z$$

and clips small negative floating-point values to zero. `kernel_matrix` dispatches linear, polynomial, RBF, and Laplacian kernels. `median_gamma` samples at most 1,000 points before computing the median nonzero squared distance.

KernelRidgeScratch. The class stores training vectors, centers the target, builds K , solves for α , and predicts with a test/train Gram matrix.

KernelSVMScratch. The class creates one output column per class. Labels are converted to $+1/-1$, squared-hinge active margins are found, and representer coefficients are optimized by Adam. The regularizer uses $K\alpha$, which corresponds to RKHS norm $\alpha^\top K\alpha$.

KernelKNNScratch. Training stores the diagonal of the training Gram matrix. Prediction computes feature-space squared distances from kernel identities and performs weighted KNN.

KernelPCAScratch. Training stores column means and global mean of K , centers it, requests the largest eigenpairs with SciPy `eigh`, removes nonpositive eigenvalues, and normalizes eigenvectors by $\sqrt{\lambda}$. Test kernels are centered against training statistics before projection.

IsolationForestScratch. Each tree recursively selects a variable with nonzero range and a random split. Leaves store sample count. The score uses average path length normalized by the binary-search-tree approximation $c(n)$. The class is used diagnostically, not as a supervised model.

Metrics and diagnostics. Confusion matrices are assembled explicitly. Classification metrics compute per-class precision/recall/F1. Kernel geometry computes alignment, eigenvalues, effective rank, and off-diagonal statistics. Worst-case functions append actual, predicted, absolute/relative error, then sort.

A.2 How to read `homework_experiments.py`

`PreparedTask` is a dataclass containing raw and encoded train/test features, model-scale and evaluation-scale targets, inverse target transformation, encoders, preprocessing object, and full engineered row count.

`_split_frame` supports three split modes:

1. group split for Airbnb hosts;
2. chronological split for NYC;
3. stratified/random split for ordinary tasks.

The code asserts that Airbnb training and test host sets are disjoint. `_prepare_from_frame` then performs controlled subsampling, fits the preprocessor only on training rows, encodes labels, and stores the inverse log-target function.

`models_for_task` is the experiment registry. Classification receives logistic, KNN, tree, three SVM kernels, two kernel-KNN variants, and RBF-KPCA logistic. Regression receives linear, KNN, tree, three KRR variants, and two kernel-KNN variants.

`run_all_experiments`:

1. computes kernel geometry on up to 260 training cases;
2. fits every registered model with `tracemalloc` and wall-clock timing;
3. converts regression predictions back to original units;
4. records metrics and predictions;
5. selects the best task model by macro-F1 or MAE;
6. writes the ten worst cases with raw engineered features;
7. saves model and geometry tables.

A.3 Reproduction procedure

1. Place the four supplied raw files in `data/`.
2. Run `ML_Homework_Complete.ipynb` from the project root.
3. The first run streams and caches NYC 311. Later runs reuse the cache unless the source file changes.
4. Inspect `processed/model_tables/` for task tables.
5. Inspect `artifacts/model_results.csv`, `kernel_geometry.csv`, and `worst_10*.csv`.
6. Compile this report from the `report/` directory with a standard LaTeX/BibTeX workflow.

A.4 Interpretation checklist

Before claiming that a model “wins,” verify:

1. Was the correct observational unit used?
2. Was the target known only after the prediction time?
3. Were identifiers or target proxies removed?
4. Were preprocessing statistics fit on training data only?
5. Was grouping or time respected in the split?
6. Does the metric expose minority and tail errors?
7. Is a kernel improvement larger than sampling variation?
8. Do the ten worst cases indicate model bias, label ambiguity, or data error?
9. Does group performance support the intended deployment population?
10. Is exact kernel computation feasible at full scale?

Answering these questions is the practical skill the homework is intended to teach.